

DBSCAN (Функция + Матрица расстояний)

Суть: Находит кластеры как области высокой плотности, отделенные областями низкой плотности. Хорошо работает с шумами и произвольной формой.

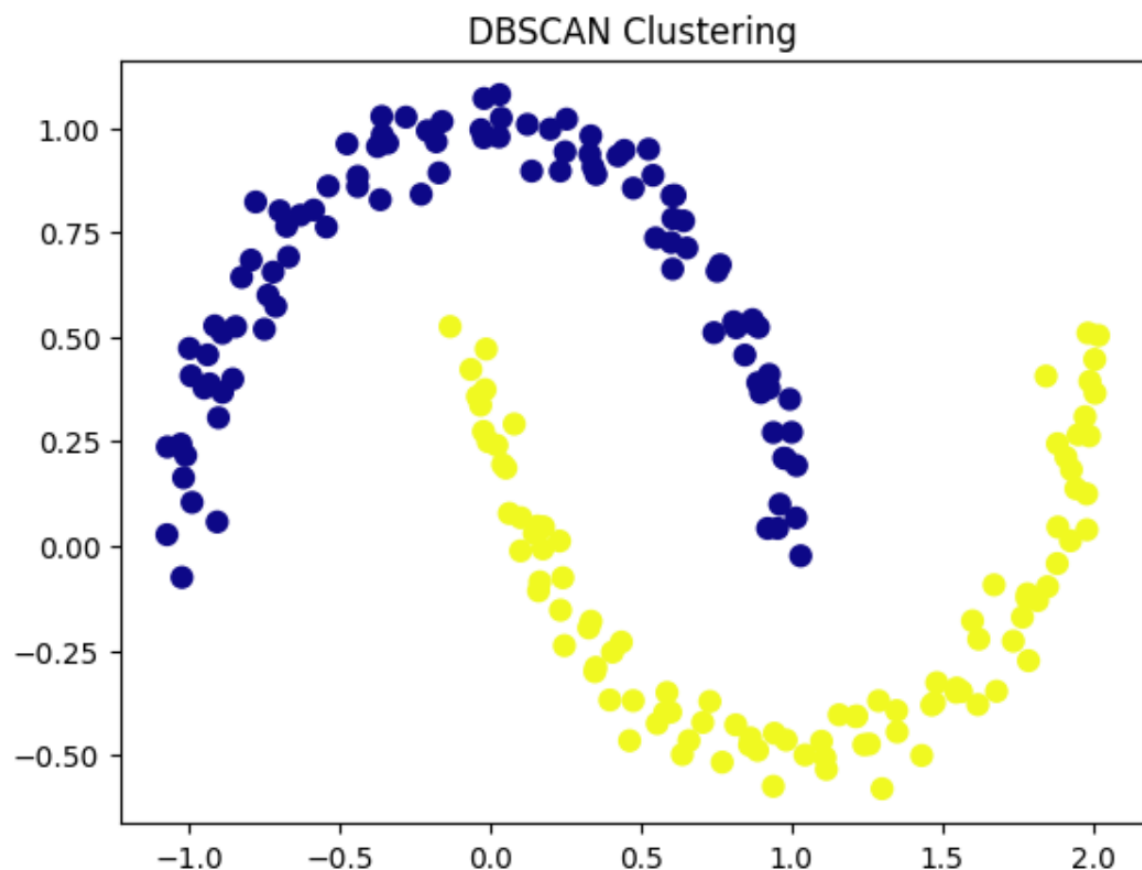
```
from sklearn.cluster import dbscan

from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# 1. Генерация данных (несферические кластеры)
X, _ = make_moons(n_samples=200, noise=0.05, random_state=0)

# 2. Запуск DBSCAN (функция)
# eps - максимальное расстояние между соседями, min_samples - мин. точек
# для ядра
core_samples, labels = dbscan(X, eps=0.3, min_samples=5)

# Визуализация
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='plasma')
plt.title("DBSCAN Clustering")
plt.show()
```



Когда использовать: Для поиска аномалий в транзакциях, кластеризации географических данных (например, координат магазинов), где кластеры могут быть произвольной формы и есть шум.

- **Параметры:** `eps` (радиус окрестности), `min_samples` (минимальное количество точек в окрестности для образования ядра).
- **Геометрия:** Расстояние между точками.
- **Особенность:** Трансдуктивный метод (не умеет предсказывать новые точки, только переобучаться), автоматически определяет количество кластеров и выбросы (метка -1).

Пример: Поиск аномалий и кластеров произвольной формы

Задача: Данные о местоположении (долгота/широта) содержат плотные скопления (районы) и редкие выбросы (отдаленные фермы).

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import DBSCAN

from sklearn.datasets import make_moons

# 1. Данные в форме полумесяцев (классика для DBSCAN)
X, _ = make_moons(n_samples=300, noise=0.1, random_state=42)

# 2. Создание модели
# eps - радиус поиска соседей (подбирается под данные)
# min_samples - чем больше, тем меньше чувствительность к шуму
model = DBSCAN(eps=0.2, min_samples=5)

# 3. Обучение (fit_predict сразу возвращает метки)
labels = model.fit_predict(X)

# 4. Анализ результатов
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)

print(f'Найдено кластеров: {n_clusters}')

print(f'Количество шумовых точек (выбросов): {n_noise}')

# 5. Визуализация
```

```

# Шумовые точки красим в черный
unique_labels = set(labels)
colors = plt.cm.Spectral(np.linspace(0, 1, len(unique_labels)))
for k, col in zip(unique_labels, colors):
    if k == -1:
        # Черный цвет для шума
        col = 'k'
    class_member_mask = (labels == k)
    xy = X[class_member_mask]
    plt.scatter(xy[:, 0], xy[:, 1], s=50, c=[col], edgecolor='k', alpha=0.7)
plt.title('DBSCAN: Обнаружение кластеров и шума')
plt.show()

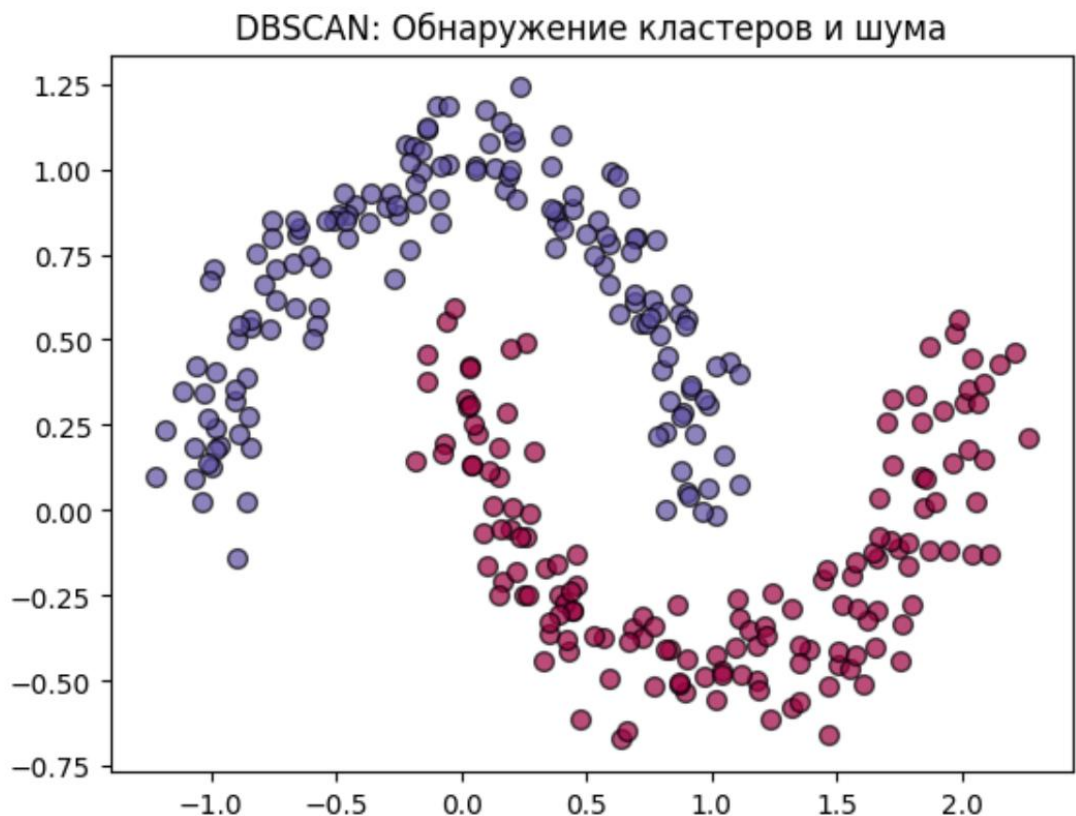
```

```

Найдено кластеров: 2
Количество шумовых точек (выбросов): 0

```

...



Проверка модели (настройка параметров): Главная сложность DBSCAN — подбор ϵ . Если взять слишком маленький ϵ , все точки станут шумом. Если слишком большой — все сольется в один кластер.

Метод K-distance: Строится график расстояний до k-го ближайшего соседа (где $k = \text{min_samples}$). Точка перегиба на этом графике — хорошее значение для eps .

```
from sklearn.neighbors import NearestNeighbors
import numpy as np

neigh = NearestNeighbors(n_neighbors=5) # min_samples = 5
nbrs = neigh.fit(X)
distances, indices = nbrs.kneighbors(X)

# Берем расстояние до 5-го соседа (индекс 4), сортируем и строим график
k_dist = np.sort(distances[:, 4])
plt.plot(k_dist)
plt.title('График K-distance для выбора eps')
plt.ylabel('Расстояние до 5-го соседа')
plt.xlabel('Точки (отсортированы)')
plt.grid(True)
plt.show()

# Ищем точку "излома" на графике – это и будет оптимальный eps
```



Задача 2: Обработка шума (DBSCAN)

Вопрос: В данных есть выбросы. Какой алгоритм выбрать и как интерпретировать результат?

Решение: Использовать DBSCAN. Точки, не попавшие в кластеры (метка -1), считаются шумом/выбросами.

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
import numpy as np

# Добавляем шум в данные
X, _ = make_blobs(n_samples=300, centers=3, random_state=42)
noise = np.random.uniform(low=-10, high=10, size=(50, 2))
X_noisy = np.vstack([X, noise])

# Кластеризация DBSCAN
db = DBSCAN(eps=1.5, min_samples=5)
labels = db.fit_predict(X_noisy)

# Анализ: сколько шума?
n_noise = list(labels).count(-1)

print(f"Количество точек, классифицированных как шум: {n_noise}")
print(f"Уникальные кластеры: {set(labels)}")
```

Количество точек, классифицированных как шум: 32

Уникальные кластеры: {np.int64(0), np.int64(1), np.int64(2), np.int64(-1)}

Объяснение: DBSCAN автоматически помечает выбросы (метка -1), не требуя заранее задавать количество кластеров. Если шума много, нужно настроить параметры eps (радиус) и min_samples.

Задача 3: Проверка качества (Метрики)

Вопрос: У нас есть размеченные данные (y_true), но мы хотим кластеризовать. Насколько хорошо сработала модель?

Решение: использовать метрики сравнивающие полученные метки с истинными (для валидации).

```
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
from sklearn.datasets import make_blobs

# Данные с истинными метками (используем для проверки, но не для обучения!)
X, y_true = make_blobs(n_samples=300, centers=4, random_state=0)

# Кластеризация (модель не видит y_true)
kmeans = KMeans(n_clusters=4, random_state=0)
y_pred = kmeans.fit_predict(X)

# Оценка
ari = adjusted_rand_score(y_true, y_pred)
nmi = normalized_mutual_info_score(y_true, y_pred)

print(f'Adjusted Rand Index (ARI): {ari:.3f}') # 1.0 - идеальное совпадение
print(f'Normalized Mutual Info (NMI): {nmi:.3f}') # 1.0 - идеальное совпадение
```

Adjusted Rand Index (ARI): 0.823. Normalized Mutual Info (NMI): 0.786

Объяснение: ARI и NMI измеряют сходство между двумя разбиениями (истинным и предсказанным). Значения близкие к 1 означают, что кластеризация почти идеально совпала с реальной структурой данных.

1. Суть метода DBSCAN (Кратко)

DBSCAN — это алгоритм кластеризации, основанный на плотности.

- **Цель:** Находит кластеры произвольной формы, отделяя их от шума.
- **Отличие от k-средних:** Не требует указания количества кластеров и может находить кластеры сложной (невыпуклой) формы.

Ключевые понятия:

1. **eps (Эпсилон):** Радиус окрестности точки.
2. **min_samples:** Минимальное количество точек, которое должно находиться внутри радиуса eps, чтобы точка считалась **ядровой**.
3. **Ядровая точка:** Точка, в окрестности которой находится не менее min_samples других точек.
4. **Граничная точка:** Точка, которая не является ядровой, но достижима из ядровой.
5. **Шум (Выброс):** Точка, не являющаяся ни ядровой, ни граничной.

Как строится кластер: Кластер формируется путем рекурсивного объединения ядерных точек, находящихся на расстоянии ϵ друг от друга, и присоединения к ним граничных точек.

2. Пример на Python (Развернутый и запущенный)

В этом примере мы сгенерируем данные сложной формы, применим DBSCAN и визуализируем результат.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import DBSCAN

from sklearn.datasets import make_moons, make_blobs

from sklearn.preprocessing import StandardScaler

# --- Генерация данных (Два полумесяца + шум) ---

X, _ = make_moons(n_samples=300, noise=0.1, random_state=42)

# Добавим немного случайного шума для наглядности

rng = np.random.RandomState(42)

noise = rng.uniform(low=-1.5, high=1.5, size=(20, 2))

X = np.vstack([X, noise])

# Стандартизация данных (важно для DBSCAN, если признаки имеют разный масштаб)

X_scaled = StandardScaler().fit_transform(X)

# --- Применение DBSCAN ---

# Параметры подобраны эмпирически для этой задачи

eps_value = 0.3

min_samples_value = 5

db = DBSCAN(eps=eps_value, min_samples=min_samples_value)

labels = db.fit_predict(X_scaled)

# --- Анализ результатов ---

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

n_noise = list(labels).count(-1)
```

```

print(f'Алгоритм: DBSCAN')
print(f'Параметры: eps={eps_value}, min_samples={min_samples_value}')
print(f'Найдено кластеров: {n_clusters}')
print(f'Количество точек шума: {n_noise}')

# --- Визуализация ---
unique_labels = set(labels)
core_samples_mask = np.zeros_like(labels, dtype=bool)
core_samples_mask[db.core_sample_indices_] = True
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
plt.figure(figsize=(10, 7))
for k, col in zip(unique_labels, colors):
    if k == -1: # Черный цвет для шума
        col = [0, 0, 0, 1]
    class_member_mask = (labels == k)
    # Ядровые точки (большие круги)
    xy = X_scaled[class_member_mask & core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=14, label=f'Кластер {k} (ядро)' if k != -1
             else 'Шум')
    # Граничные точки (маленькие круги)
    xy = X_scaled[class_member_mask & ~core_samples_mask]
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=6)
plt.title(f'Кластеризация DBSCAN\nНайдено кластеров: {n_clusters}')
plt.xlabel('Признак 1 (стандартизованный)')
plt.ylabel('Признак 2 (стандартизованный)')
plt.legend()
plt.grid(True)
plt.show()

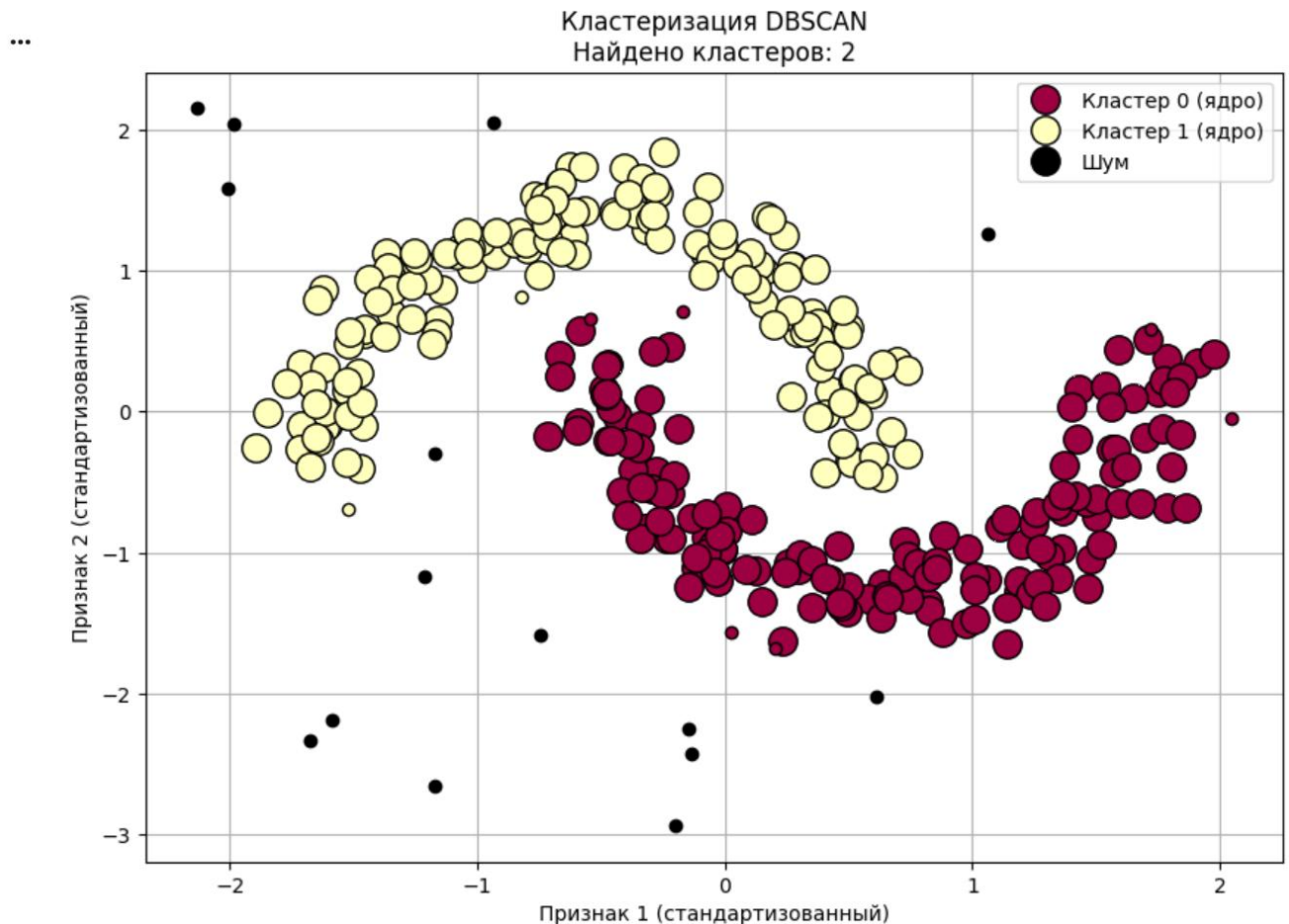
```


Алгоритм: DBSCAN

Параметры: $\text{eps}=0.3$, $\text{min_samples}=5$

Найдено кластеров: 2

Количество точек шума: 15



На графике видно 2 отчетливых кластера (полумесяца), а точки, случайно разбросанные в стороне, будут помечены черным цветом как шум.

Разбор кода:

Импорт библиотек: `numpy`, `matplotlib`, `DBSCAN` из `sklearn.cluster`, функции генерации данных `make_moons`. **Генерация данных:**

`make_moons` создает 2 полумесяца (хороший пример невыпуклых кластеров)

Мы добавляем искусственный шум, чтобы проверить, как `DBSCAN` идентифицирует выбросы.

`StandardScaler` — обязательный шаг. `DBSCAN` использует расстояние (обычно евклидово). Если признаки имеют разный масштаб (например, рост в см и вес в кг), то признак с большими значениями будет доминировать. Стандартизация приводит все признаки к единому масштабу.

Инициализация и обучение: DBSCAN($\text{eps}=0.3$, $\text{min_samples}=5$) — создаем модель с параметрами.

`.fit_predict(X_scaled)` — обучается и сразу возвращает метки кластеров для каждой точки. Метка -1 означает "шум".

Анализ: `db.core_sample_indices_` — специальный атрибут, хранящий индексы ядровых точек. Мы используем его для визуализации (большие кружки).

Подсчет уникальных меток (исключая -1) дает количество кластеров.

Визуализация: Разным цветами отрисовываются разные кластеры. Ядровые точки рисуются большими кругами, граничные — маленькими, шум — черным.

Задачи для проверки модели (с объяснением)

Вот несколько задач, которые помогут проверить понимание DBSCAN, и их решения. **Задача 1: Влияние параметра eps**

Условие: У вас есть набор данных. Вы запустили DBSCAN с $\text{min_samples}=5$. При $\text{eps}=0.2$ все точки оказались шумом (помечены -1). При $\text{eps}=2.0$ все точки попали в один кластер. Какую проблему это иллюстрирует и что делать?

Решение: Проблема: Неверно выбран параметр eps .

$\text{eps}=0.2$ слишком мал. В окрестности радиуса 0.2 не набирается 5 соседей ни для одной точки, поэтому все точки считаются шумом.

$\text{eps}=2.0$ слишком велик. Окрестности точек перекрываются настолько сильно, что алгоритм связывает все точки в одну структуру.

Что делать: Нужно подобрать оптимальный eps . Для этого часто строят график **k-расстояний** (k-distance graph).

1) Для каждой точки вычисляют расстояние до её k-го ближайшего соседа (где $k = \text{min_samples}$).

2) Сортируют эти расстояния по убыванию и строят график.

3) Ищут точку "изгиба" (резкого изменения кривой) — значение eps на этой точке считается оптимальным.

Задача 2: Кластеризация неравномерной плотности

Условие: На ваших данных есть кластеры с разной плотностью (один очень плотный, другой разреженный). DBSCAN с фиксированным eps либо разбивает разреженный кластер на шум и мелкие куски, либо сливает плотный кластер с соседями. Как быть?

Решение:

Это классическое ограничение DBSCAN. С 1 глобальным ϵ он не может корректно обработать кластеры разной плотности.

Варианты решения:

Использовать алгоритм **OPTICS** (Ordering Points To Identify the Clustering Structure). Он является расширением DBSCAN и не требует жесткого задания ϵ . Он создает порядок кластеризации, из которого можно извлечь кластеризацию для любого ϵ .

Попробовать другие алгоритмы кластеризации на основе плотности, которые решают эту проблему, например, **HDBSCAN**.

Задача 3: Неправильная классификация граничных точек

Условие: В документации к `sklearn` сказано, что DBSCAN не детерминирован для граничных точек. У вас есть неосновная точка, которая находится на расстоянии ϵ от двух разных кластеров. В одном запуске она попала в кластер А, в другом — в кластер В. Почему?

Решение: это не баг, а особенность реализации, описанная в тексте.

Неосновная (граничная) точка находится на равном удалении от двух ядровых точек из разных кластеров. По определению, два ядра из разных кластеров не могут быть на расстоянии меньше ϵ друг от друга (иначе они были бы в одном кластере). Однако граничная точка может находиться ровно на границе.

Алгоритм DBSCAN обрабатывает данные в том порядке, в котором они идут. Когда он доходит до этой граничной точки, он присваивает ей метку того кластера, который был найден **раньше** в процессе обработки. Порядок данных влияет на то, какой кластер будет обработан первым, а значит, и на метку граничной точки. Это единственный источник недетерминизма в алгоритме.